

Python – Syntax & Kontrollstrukturen

if · Schleifen · break/continue · pass · try-except – je ein Beispiel

0. Grundlagen der Python-Syntax

Einrückung statt Klammern: Blöcke werden durch Einrückung (Standard: 4 Leerzeichen) gebildet. Falsche Einrückung führt zu einem **IndentationError**.

Doppelpunkt: Jede Kontrollstruktur (**if**, **for**, **while**, **try**, **def** ...) endet mit einem **:**, danach folgt der eingerückte Block.

Kommentare beginnen mit **#**. Keine Semikolons am Zeilenende nötig.

```
alter = 17                # Variable, kein Datentyp nötig
if alter >= 18:           # Doppelpunkt!
    print("Volljährig")   # eingerückt = gehört zum if
print("Fertig")           # nicht eingerückt = außerhalb
```

1. Verzweigung: if / elif / else

Bedingungen werden von oben nach unten geprüft. Nur der **erste zutreffende** Block wird ausgeführt. **elif** und **else** sind optional.

```
note = 2

if note == 1:
    print("Sehr gut")
elif note == 2:
    print("Gut")
elif note in (3, 4):
    print("Befriedigend oder Genügend")
else:
    print("Nicht genügend")
```

Vergleichsoperatoren: **==** **!=** **<** **>** **<=** **>=** **in** · Logische Operatoren: **and** **or** **not**

2. Schleifen: for und while

for iteriert über eine bekannte Menge (Liste, String, **range()**). **while** läuft, solange eine Bedingung wahr ist – Achtung auf Endlosschleifen!

```
# for-Schleife über eine Liste
noten = [1, 3, 2, 5]
for n in noten:
    print("Note:", n)

# for mit range(start, stop, schritt) -> 0, 1, 2, 3, 4
for i in range(5):
    print(i)

# while-Schleife
versuche = 0
while versuche < 3:
    print("Versuch", versuche + 1)
    versuche += 1          # sonst Endlosschleife!
```

3. break und continue

break beendet die Schleife **sofort komplett**. **continue** überspringt nur den **aktuellen Durchlauf** und macht mit dem nächsten weiter.

```
noten = [1, 3, 2, 5, 2]

for n in noten:
    if n == 5:
        print("Nicht genügend gefunden -> Abbruch")
        break          # Schleife endet, letzte 2 wird nie erreicht
    print("OK:", n)

for n in noten:
    if n == 3:
        continue       # 3 wird übersprungen, Schleife läuft weiter
    print(n)           # Ausgabe: 1 2 5 2
```

4. pass - der Platzhalter

Python verlangt in jedem Block mindestens eine Anweisung. **pass** erfüllt diese Regel, tut aber **nichts**. Typisch für Funktionen/Klassen, die erst später implementiert werden, oder für Fälle, die bewusst ignoriert werden.

```
def zeugnis_drucken():
    pass                # TODO: später implementieren

for n in [1, 0, 3]:
    if n == 0:
        pass           # 0 = nicht angetreten, bewusst ignorieren
    else:
        print(n)
```

Merke: **pass** = nichts tun, Schleife läuft normal weiter. **continue** = Rest des Durchlaufs überspringen. **break** = Schleife verlassen.

5. Fehlerbehandlung: try / except / else / finally

Statt dass das Programm bei einem Fehler abstürzt, wird der Fehler abgefangen und behandelt.

Block	Wird ausgeführt ...
<code>try</code>	immer – enthält den Code, der einen Fehler auslösen könnte
<code>except</code>	nur wenn ein Fehler auftritt (mehrere für verschiedene Fehlertypen möglich)
<code>else</code>	nur wenn kein Fehler aufgetreten ist (optional)
<code>finally</code>	immer – auch bei return, break oder continue (optional, z. B. zum Aufräumen)

```
try:
    summe = int(input("Summe der Noten: "))
    schnitt = summe / anzahl
except ValueError:
    print("Bitte eine ganze Zahl eingeben!")
except ZeroDivisionError:
    print("Division durch 0 nicht möglich!")
else:
    print("Schnitt:", schnitt)
finally:
    print("Berechnung abgeschlossen.")
```

Häufige Fehlertypen: `ValueError` (falscher Wert, z. B. `int("abc")`), `ZeroDivisionError`, `TypeError`, `IndexError`, `KeyError`, `FileNotFoundError`

6. Gesamtbeispiel: Bankomat-Simulation

Alle Kontrollstrukturen in einem Programm. Test-PIN: **1234**.

Struktur	Einsatz im Bankomat
<code>for + break</code>	max. 3 PIN-Versuche, Abbruch bei richtiger PIN
<code>while + break</code>	Hauptmenü läuft bis „4 = Beenden“
<code>if/elif/else</code>	Menüauswahl und Prüfung des Betrags
<code>try/except/else/finally</code>	ungültige Beträge abfangen, „Abhebevorgang beendet“ immer ausgeben
<code>continue</code>	nach Fehleingabe zurück ins Menü
<code>pass</code>	Platzhalter-Funktion <code>kontoauszug_drucken()</code>

```
"""
Bankomat-Simulation – Übungsbeispiel für Python-Kontrollstrukturen
Themen: if/elif/else, for, while, break, continue, pass,
        try-except-else-finally

Test-PIN: 1234
"""

RICHTIGE_PIN = "1234"
kontostand = 500.0

def kontoauszug_drucken():
    pass # PASS: Funktion ist noch nicht implementiert

# ----- 1. PIN-Eingabe: FOR + BREAK (max. 3 Versuche) -----
angemeldet = False
for versuch in range(1, 4):
    pin = input(f"PIN eingeben (Versuch {versuch}/3): ")
    if pin == RICHTIGE_PIN:
        angemeldet = True
        break # richtige PIN -> keine weiteren Versuche nötig
    print(" Falsche PIN!")

if not angemeldet:
    print("Karte wurde eingezogen. Bitte wenden Sie sich an Ihre Bank.")
    exit()
```

```

# ----- 2. Hauptmenü: WHILE-Schleife -----
while True:
    print("\n1 = Kontostand | 2 = Abheben | 3 = Kontoauszug | 4 = Beenden")
    auswahl = input("Auswahl: ")

    # IF / ELIF / ELSE: Menüauswahl auswerten
    if auswahl == "1":
        print(f"Kontostand: {kontostand:.2f} €")

    elif auswahl == "2":
        # TRY-EXCEPT-ELSE-FINALLY: ungültige Beträge abfangen
        try:
            betrag = float(input("Betrag in €: "))
        except ValueError:
            # läuft nur, wenn ein Fehler auftritt
            print(" Fehler: Bitte eine Zahl eingeben!")
            continue # zurück zum Menü (finally läuft trotzdem!)
        else:
            # läuft nur, wenn KEIN Fehler aufgetreten ist
            if betrag <= 0:
                print(" Fehler: Betrag muss positiv sein!")
            elif betrag % 10 != 0:
                print(" Fehler: Nur 10-€-Schritte möglich!")
            elif betrag > kontostand:
                print(" Fehler: Kontostand nicht ausreichend!")
            else:
                kontostand -= betrag
                print(f" {betrag:.2f} € ausgezahlt. Neuer Stand: {kontostand:.2f} €")
        finally:
            # läuft IMMER – mit oder ohne Fehler
            print(" Abhebevorgang beendet.")

    elif auswahl == "3":
        kontoauszug_drucken()
        print(" Diese Funktion ist bald verfügbar.")

    elif auswahl == "4":
        print("Auf Wiedersehen!")
        break # verlässt das Hauptmenü

    else:
        print(" Ungültige Auswahl!")

```